Name : Shivraj Vaibhav Jagtap
Roll No : EN24107060
Class : SY - AI \& DS
Subject : Python for Visual AI

Assignment 8:

A startup wants historical rainfall data stored for analysis from https://www.data.gov.in/resources/rainfall-india Scrape tabular rainfall data, Clean missing values, Store data in: CSV and Reload data for analysis

---

Step 1: Importing the necessary python libraries

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Loading the dataset into pandas dataframe

In [2]:
```
df = pd.read_csv("rainfall in india 1901-2015.csv")

df.head()
```

Out[2]:

	SUBDIVISION	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP
0	ANDAMAN & NICOBAR ISLANDS	1901	49.2	87.1	29.2	2.3	528.8	517.5	365.1	481.1	332.6
1	ANDAMAN & NICOBAR ISLANDS	1902	0.0	159.8	12.2	0.0	446.1	537.1	228.9	753.7	666.2
2	ANDAMAN & NICOBAR ISLANDS	1903	12.7	144.0	0.0	1.0	235.1	479.9	728.4	326.7	339.0
3	ANDAMAN & NICOBAR ISLANDS	1904	9.4	14.7	0.0	202.4	304.5	495.1	502.0	160.1	820.4
4	ANDAMAN & NICOBAR ISLANDS	1905	1.3	0.0	3.3	26.9	279.5	628.7	368.7	330.5	297.0

Step 3 : Data Preprocessing

In [3]:
```
df.info()
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4116 entries, 0 to 4115
Data columns (total 19 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   SUBDIVISION  4116 non-null   object 
 1   YEAR         4116 non-null   int64  
 2   JAN          4112 non-null   float64
 3   FEB          4113 non-null   float64
 4   MAR          4110 non-null   float64
 5   APR          4112 non-null   float64
 6   MAY          4113 non-null   float64
 7   JUN          4111 non-null   float64
 8   JUL          4109 non-null   float64
 9   AUG          4112 non-null   float64
 10  SEP          4110 non-null   float64
 11  OCT          4109 non-null   float64
 12  NOV          4105 non-null   float64
 13  DEC          4106 non-null   float64
 14  ANNUAL       4090 non-null   float64
 15  Jan-Feb      4110 non-null   float64
 16  Mar-May      4107 non-null   float64
 17  Jun-Sep      4106 non-null   float64
 18  Oct-Dec      4103 non-null   float64
dtypes: float64(17), int64(1), object(1)
memory usage: 611.1+ KB
```

Out[3]:
```
SUBDIVISION     0
YEAR            0
JAN             4
FEB             3
MAR             6
APR             4
MAY             3
JUN             5
JUL             7
AUG             4
SEP             6
OCT             7
NOV            11
DEC            10
ANNUAL         26
Jan-Feb         6
Mar-May         9
Jun-Sep        10
Oct-Dec        13
dtype: int64
```

In [4]:
```
# Replace empty values with NaN
df.replace('', pd.NA, inplace=True)

# Convert only numeric columns
numeric_cols = df.columns[1:]
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric, errors='coerce')

# Fill missing numeric values with mean
df.fillna(df.mean(numeric_only=True), inplace=True)

df.isnull().sum()
```

Out[4]:
```
SUBDIVISION    0
YEAR           0
JAN            0
FEB            0
MAR            0
APR            0
MAY            0
JUN            0
JUL            0
AUG            0
SEP            0
OCT            0
NOV            0
DEC            0
ANNUAL         0
Jan-Feb        0
Mar-May        0
Jun-Sep        0
Oct-Dec        0
dtype: int64
```

Step 4 : Print the numneric values of mean of each column

In [5]:
```
print("Average Rainfall:\n")
print(df.mean(numeric_only=True))
```

```
Average Rainfall:

YEAR        1958.218659
JAN           18.957320
FEB           21.805325
MAR           27.359197
APR           43.127432
MAY           85.745417
JUN          230.234444
JUL          347.214334
AUG          290.263497
SEP          197.361922
OCT           95.507009
NOV           39.866163
DEC           18.870580
ANNUAL      1411.008900
Jan-Feb       40.747786
Mar-May      155.901753
Jun-Sep     1064.724769
Oct-Dec      154.100487
dtype: float64
```

Step 5 : State-wise Rainfall Analysis

We first check if the dataset contains a **State** column.
If present, we compute the **average rainfall values** for each state.

$$\text{state_avg} = \text{groupby}(\text{State}) \rightarrow \text{mean}$$

In [6]:
```
if 'State' in df.columns:
    state_avg = df.groupby('State').mean(numeric_only=True)
    print(state_avg)
```

Step 6 : Reloading Dataset

Reloading the Cleaned Dataset

We reload the saved dataset to ensure persistence and correctness.

In [7]:
```
df.to_csv("cleaned_rainfall_data.csv", index=False)
print("Cleaned dataset saved!")
```

```
Cleaned dataset saved!
```

In [8]:
```
df_reloaded = pd.read_csv("cleaned_rainfall_data.csv")

df_reloaded.head()
```

Out[8]:

	SUBDIVISION	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP
0	ANDAMAN & NICOBAR ISLANDS	1901	49.2	87.1	29.2	2.3	528.8	517.5	365.1	481.1	332.6
1	ANDAMAN & NICOBAR ISLANDS	1902	0.0	159.8	12.2	0.0	446.1	537.1	228.9	753.7	666.2
2	ANDAMAN & NICOBAR ISLANDS	1903	12.7	144.0	0.0	1.0	235.1	479.9	728.4	326.7	339.0
3	ANDAMAN & NICOBAR ISLANDS	1904	9.4	14.7	0.0	202.4	304.5	495.1	502.0	160.1	820.4
4	ANDAMAN & NICOBAR ISLANDS	1905	1.3	0.0	3.3	26.9	279.5	628.7	368.7	330.5	297.0

Step 7 : Mean Calculation and Visualization

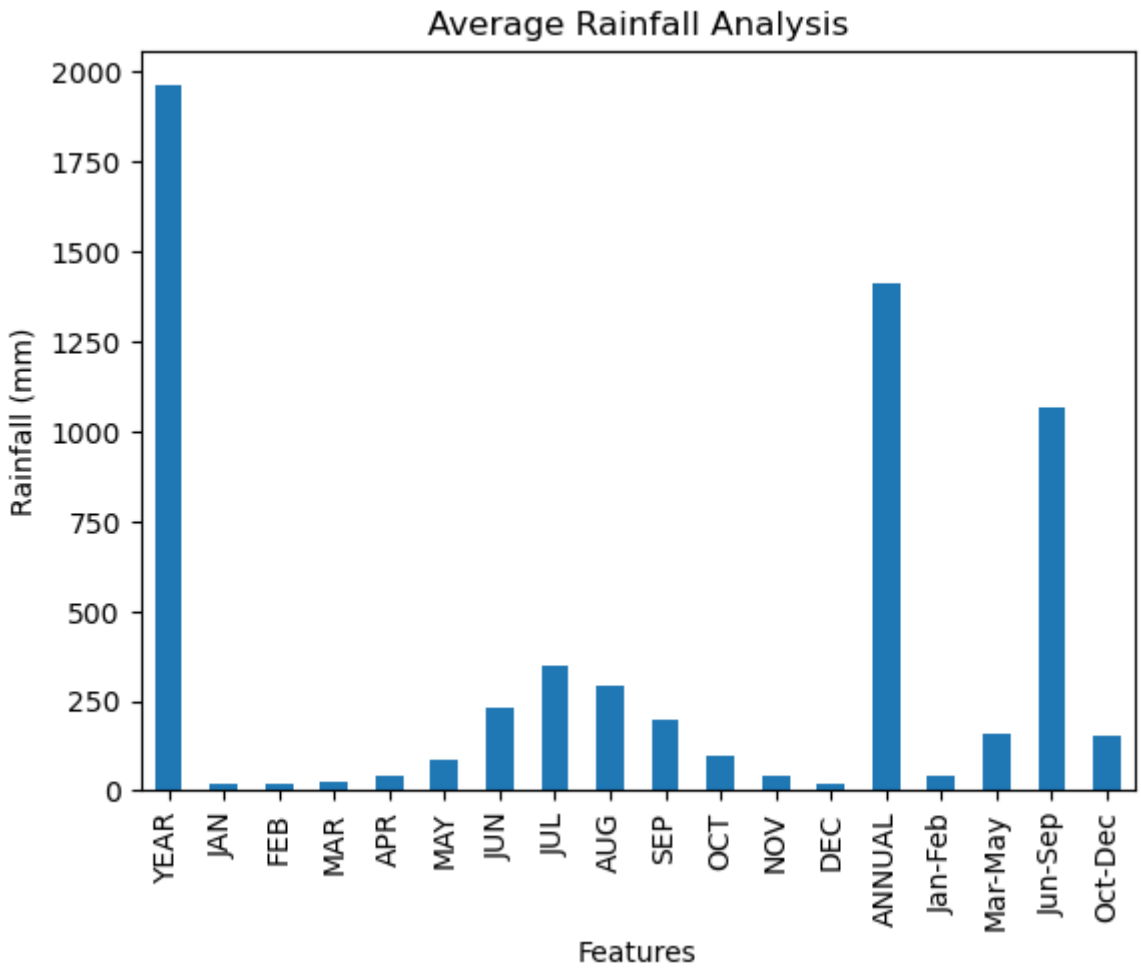
Rainfall Feature Analysis

We compute the **mean rainfall values** across all numeric features:

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i$$

In [9]:
```
df_reloaded.mean(numeric_only=True).plot(kind='bar')

plt.title("Average Rainfall Analysis")
plt.xlabel("Features")
plt.ylabel("Rainfall (mm)")
plt.show()
```



Conclusion

In this lab exercise, we learned how to extract tabular data from an online source using web scraping, clean the dataset to handle missing and inconsistent values, store the cleaned data into a CSV file, and reload it for further usage. This process is widely used in real-world data science for gathering datasets from public portals that do not provide direct APIs or downloads.